CS175 Project Proposal

Atharv Attri	Adhith Jacob	Shreya Nakum
aattri@uci.edu	adhithj@uci.edu	snakum@uci.edu
71872115	66072553	17143467

July 26, 2026

1 Introduction

Prediction markets such as Kalshi and Polymarket allow users to buy binary contracts that pay one dollar if an event resolves `YES` and nothing if it resolves `NO`, which means the price of the contract functions as a market-implied probability of that event. Our project targets the subset of these markets that are tied to cryptocurrency price levels, meaning contracts of the form "Will BTC be above $118,000 at 5:00 PM ET today?" We focus on these because they resolve on fifteen-minute horizons, which produces thousands of complete episodes per day, and because they settle against a public and verifiable reference price rather than a subjective judgment call.

The task itself is one of sequential decision-making under uncertainty, because given a live market that has not yet resolved, the system must decide at each timestep whether to take a position, close one, or do nothing, with the objective of maximizing risk-adjusted profit net of trading fees. From the point of view of the end user, the system is best understood as a trading agent. It ingests a live or replayed market feed and emits an action on every tick, and the user supplies a capital budget and a market universe, while the system returns a position log, a running PnL curve, and a per-decision explanation of why it acted. It is important to note that we are not building a general forecaster, but rather a system that decides when the market's own price is wrong enough to be worth trading against, and worth the fees.

This distinction matters because the cost of trading here is large and highly structured. Polymarket charges takers $fee = C \cdot r \cdot p(1-p)$, where C is share count and p is price; the crypto category carries $r = 0.07$, the highest rate on the platform. At $p = .50$ that is 1.75¢ per share, so a round trip at the money costs about 3.5¢ of probability before the spread. Fees also shrink toward the extremes (.63¢ at $p = .10$), and redemption at settlement is free, so holding to resolution is strictly cheaper than exiting early. A model can therefore be better calibrated than the market and still lose money on every trade it makes. The trading policy is a separate learning problem from the probability estimate, and that separation is the main idea behind our project.

2 Input and Outputs

The state space at each timestep will consist of the current `YES` and `NO` prices (probability), the recent price of the crypto, any indicators derived from the price that we find to be valuable, trading volume, and time remaining until resolution. We considered using external information like news and polling data but due to the fifteen-minute resolution horizon of our target market, we expect price to dominate the slower-moving external information.

The output will be a discrete class: Buy `YES`, buy `NO`, `hold`, or `sell`.

Our dataset is gathered through the Polymarket API and Jon Becker's Prediction Market Analysis to query directly for tick-level price and trade history on the bitcoin threshold markets. We plan to have a uniform schema across both datasets so that episodes can be merged into a single training set regardless of data origin.

The main reason we decided to focus on Polymarket instead of Kalshi is because Kalshi gets the underlying prices from CF Benchmarks, which have their own proprietary algorithm to price the

cryptocurrency and does not have a public API. Polymarket uses Chainlink BTC/USD CEX price streams, which does have a public API. This allows us to ensure that we are always trading with the price that is actually correct.

3 Approach

We will be developing two baseline models plus a trivial no trade floor and a reinforcement learning model that will be trained with Q-learning.

3.1 Baseline 1: XGBoost

We will train an XGBoost model to predict the probability that a market resolved `YES` given the current features. This model will not make trading decisions independently and will only estimate the "true" probability, which we can then compare against the market's price. We chose XGBoost because it handles categorical and continuous features well and trains quickly. To compare this model against the RL agent on trading metrics, we pair it with a fixed-threshold decision rule: take a position whenever $|\hat{p}_t - p_t^{mkt}| > \theta$, with θ selected on the validation split. This gives the RL agent a policy that consumes the same signal and differs only in that its entry rule is a tuned constant rather than a learned, state-dependent function.

3.2 Baseline 2: Buy and Hold Model

This model takes a position when the market opens based solely on the initial price and holds until resolution. This model will serve as a sanity checker, as any model should be able to outperform a model that just accepts a market's initial price.

3.3 Reinforcement Learning Model

We will treat each market's price path, from start to finish, as an episode. At each time step, the agent will observe the state described in Section 2 and chooses an action. We will use Q-learning to learn a policy over this space.

The reward at each step will be the change in mark-to-market value, a method of valuing assets at their current market price instead of their original price, minus any trading fees. At market resolution, the agent will get its money (one or zero dollars) minus the initial cost of its position. This system punishes excessive trading which addresses our concern in Section 1 that a good probability estimate doesn't mean we will have a profitable policy.

4 Evaluation Plan

Our splitting procedure is strictly temporal, because we train on markets that resolve before a cutoff date, validate on the block that immediately follows, and test on the final block; a simple train-validate-test split. No market ever appears in more than one split, and no feature is computed using information from after its own timestep. Lookahead bias is the single most likely way this project produces a fake result, which is why the data loader itself would enforce the cutoff rather than relying on us to be careful by hand.

The quantitative metrics fall into two groups. For the probability model we evaluate log loss, Brier score, the calibration curve and expected calibration error, and AUC, since these capture how accurate and well-calibrated the forecasts are. For the trading policy we look at cumulative PnL per one thousand dollars of deployed capital, the Sharpe ratio, win rate, maximum drawdown, turnover, total fees paid as a fraction of gross PnL, and average holding period, because a real-world policy is judged not only on how much it earns but on how much risk and cost it takes on to earn it.

We compare against three baselines. The first is the no-trade baseline, which produces zero PnL and serves as the honest floor, since most trading strategies lose to it. The second is buy-and-hold, which takes the favored side at open and holds every market to resolution. The third is the XGBoost model paired with its tuned threshold rule.

In terms of expected improvement, we are targeting the third baseline and aiming to beat it on the aforementioned targets. Beating this baseline would show that state-dependent timing adds value beyond a fixed edge threshold. We treat any strategy that is net-positive after fees on the held-out period as a successful outcome, given that these markets are reasonably efficient to begin with. We evaluate the probability model against the market price treated as a competing forecaster, since the question is not whether our estimate is accurate in absolute terms but whether it is more accurate than the price we would be trading against.

Finally, our moonshot is to run the trained agent in paper-trading mode against the live Polymarket feed for some amount of time and to report real-time, fully out-of-sample PnL.

5 Project Milestones

1. Week 1-2: Build the dataset pipeline for getting and cleaning historical Polymarket data.
2. Week 3: Build the simulation environment, including the fee and slippage modeling, that the RL will be trained and evaluated against.
3. Week 4: Implement and evaluate the XGBoost and buy and hold baseline models.
4. Week 5: Train and tune the Q-learning agent and run comparisons against the baseline.
5. Week 6: Finalize evaluation results and write the paper and presentation.